PARMS

Smart Parking Management System

System Audit | Architecture Proposal | ML Roadmap

Version 2.0

May 2025 | PARMS Development Team | Confidential

— Table of Contents

01 Executive Summary

PARMS (Parking and Resource Management System) is a Django-based web application built approximately two years ago to manage parking lots, vehicles, tickets, and payments for both parking authorities and drivers. The system has a solid functional foundation but lacks real-time intelligence, machine learning capabilities, and the scalability required for modern smart city deployments.

This report presents four key deliverables:

1. A full audit of the existing system — what works, what is missing, what was broken and fixed

2. A proposed architecture for PARMS v2.0 with ML integration across three system layers

3. Technology recommendations grounded in open-source and production-proven tools

4. A phased 12-month implementation roadmap with cost estimates and KPIs

The upgraded system is designed to serve two primary audiences: **parking management authorities** who need operational dashboards with real-time visibility and ML-assisted automation, and **drivers** who need fast, reliable access to parking availability, navigation, and booking — all from a web or mobile interface.

02 Current System Audit

2.1 Technology Stack (As-Is)

Component	Detail
Backend	Django 5.1 (Python), Gunicorn WSGI server
Database	PostgreSQL (production) · SQLite (local development)
Frontend	Bootstrap 5.3, vanilla JavaScript, Font Awesome 6
Authentication	Django built-in session auth with role-based redirect
Static Files	WhiteNoise middleware (CompressedManifestStaticFilesStorage in production)
Deployment	Render.com PaaS — web service + managed PostgreSQL
PDF Generation	ReportLab library
QR Codes	qrcode Python library
Missing	No REST API · No WebSockets · No ML/AI · No mobile app · No real-time features

2.2 Implemented Features

Feature	Status	Notes
User Authentication	✓ Done	Email-based login, role-based redirect (admin → dashboard, user → userin)
Admin Dashboard (SPA)	✓ Done	8-tab single-page app — no page reloads, sidebar navigation, live clock
Parking Lot CRUD	✓ Done	Create / edit / delete lots; auto-generates ParkingSpace records
Vehicle Registration	✓ Done	Plate number, vehicle type (Car/Motorcycle/Truck), owner name
Ticket Lifecycle	✓ Done	Entry time, exit time, fee calculation; space freed on ticket close
QR Code Generation	✓ Done	Per-ticket QR code generated on demand via /generate-qr/
PDF Ticket Download	✓ Done	Basic ReportLab PDF with ticket details
Payment Recording	✓ Done	Cash, Credit Card, Digital Wallet; linked to ticket
Contact Form	✓ Done	Saves to ContactMessage model; admin can view and delete
Role-based Access Control	✓ Done	_admin_required decorator on all admin views; non-staff redirected
Responsive UI	✓ Done	Bootstrap 5, mobile-friendly, Inter font, natural green theme
User Dashboard	✓ Done	Shows own vehicles, active tickets, booking history

Billings Overview	✓ Done	Revenue stats, payment history, filter by payment method
Manual Occupancy Tracking	■ Partial	is_occupied flag updated manually; no sensor or camera automation

2.3 Missing Features

Feature	Priority	Impact
License Plate Recognition (ANPR)	Critical	Eliminates manual vehicle entry — highest-impact ML feature
Real-time Occupancy Detection	Critical	Core smart parking requirement; currently fully manual
Predictive Occupancy Analytics	High	Demand forecasting to optimise pricing and space allocation
WebSocket / Real-time Updates	High	Live space availability for drivers without page refresh
REST API (mobile / IoT)	High	Required for mobile app, sensor integration, third-party services
Push / Email / SMS Notifications	High	Booking confirmation, expiry warnings, availability alerts
Map Integration & GPS Navigation	High	Driver navigation to specific parking space
Dynamic / Smart Pricing	Medium	Revenue optimisation based on occupancy and time-of-day
Automated Entry/Exit Gate Control	Medium	Hardware barrier integration via relay/IoT
Mobile App (iOS / Android)	Medium	Driver-facing native experience
Audit Logs & Security Events	Medium	Compliance and operational accountability trail
Rate Limiting & API Security	Medium	Production security — prevent abuse of public endpoints
Advanced Analytics & Charts	Medium	Visual revenue, occupancy, and usage trend charts
Export (Excel / CSV / PDF)	Low	Operational reporting for management
Multi-language Support	Low	Accessibility for non-English users

2.4 Critical Bugs & Logic Issues Found During Audit

Issue	Severity	Fix Applied
Ticket closure did not free parking space	High	Fixed in views.py: is_occupied=False + lot.available_spaces recalculated on exit
Regular users could access admin via /settings/	High	Fixed: _admin_required decorator applied to all admin views; non-staff redirected
Multiple Django servers running simultaneously on port 8000	High	Diagnosed and killed 3 stale Python processes; .env file added for correct startup

CSS never updated (CompressedManifestStaticFilesStorage in dev mode)	Medium	Fixed: settings.py now uses StaticFilesStorage when DEBUG=True
Hardcoded absolute file paths in generate_ticket()	Medium	Fixed: removed user-specific desktop path; uses relative paths
Contact form name mismatch (HTML: name='name', view expected 'full_name')	Medium	Fixed: contact_view accepts both 'name' and 'full_name' POST keys
Billings/Parkings templates used .main-content CSS class that no longer exists	Medium	Fixed: rewrote both templates to use new dashboard CSS classes
Ticket page fully hardcoded with static user 'Happy Tumukunde'	High	Fixed: ticket.html now shows logged-in user's real active tickets and history

03 Proposed System Architecture (v2.0)

3.1 Architecture Overview — 3-Layer Model

Layer 1 — Data Collection (Edge)

- IP cameras at entry/exit points stream MJPEG or RTSP video
- Ultrasonic / magnetic loop sensors detect individual space occupancy
- RFID / NFC readers at entry/exit gates for registered vehicles
- Edge compute node (Raspberry Pi 4 or NVIDIA Jetson Nano) runs lightweight YOLOv8-nano inference locally
- All sensor events published to MQTT broker (Mosquitto or AWS IoT Core) every 2 seconds

Layer 2 — Processing & Intelligence (Backend)

- Django REST Framework API handles all business logic and data persistence
- Django Channels (ASGI) + Daphne provides WebSocket connections for real-time occupancy updates
- Celery + Redis handles async tasks: ML inference jobs, email/SMS notifications, scheduled retraining
- ML microservice (FastAPI + PyTorch/ONNX) exposes high-throughput ANPR and occupancy inference endpoints
- PostgreSQL 16 stores all operational data; Redis 7 caches real-time occupancy state

Layer 3 — Presentation (Frontend & Mobile)

- Admin Dashboard: existing SPA enhanced with live WebSocket data, Chart.js visualisations, and ML controls
- Driver Web App: real-time Leaflet.js map showing available spaces, booking workflow, and navigation
- Mobile App: React Native (iOS + Android) consuming the REST API via JWT authentication
- Notifications: Firebase FCM for push, SendGrid for email, Africa's Talking for SMS

3.2 ML Component Architecture

Component A — License Plate Recognition (ANPR)

Technology: YOLOv8 (vehicle detection) + EasyOCR (text extraction)

Pipeline:

1. Camera captures frame at entry/exit point (triggered by motion sensor)
2. YOLOv8 detects vehicle bounding box; crops licence plate region of interest
3. EasyOCR extracts plate text; normalised to uppercase, no spaces
4. Django API receives plate with confidence score → looks up or creates Vehicle record
5. Ticket created automatically on entry; closed automatically on exit
6. Fallback: if confidence < 0.70, attendant prompted for manual confirmation
7. Cloud fallback: Plate Recognizer API (2,500 free lookups/month, >98% accuracy)

Target: >94% accuracy in daylight · >85% with IR cameras at night

Component B — Real-time Occupancy Detection

Technology: YOLOv8-nano (edge-optimised) + OpenCV · OR ultrasonic sensors (HC-SR04)

Pipeline:

1. Method A — Camera: overhead fisheye camera covers entire lot; each space defined as polygon
2. If vehicle centroid inside polygon for >3 seconds → space marked occupied
3. If no vehicle in polygon for >5 seconds → space marked free
4. Method B — Sensor: HC-SR04 ultrasonic per space; distance <50cm = occupied, >150cm = free
5. Sensor publishes to MQTT every 2 seconds; Celery worker subscribes and updates DB
6. WebSocket broadcasts occupancy change to all connected clients within <2 seconds

Target: >97% accuracy (camera-based, standard conditions)

Component C — Predictive Occupancy Analytics

Technology: Facebook Prophet (time-series) + scikit-learn (feature engineering)

Pipeline:

1. Trained on ParkingTicket history: entry_time, exit_time, fee, lot, day_of_week, hour
2. Predicts hourly occupancy % for each lot for the next 24 hours
3. Identifies weekly peak hours and low-demand windows for pricing guidance
4. Model retrained weekly via Celery beat task; accuracy tracked in MLPrediction model
5. Predictions stored in Redis; surfaced in admin dashboard as '24h forecast' chart
6. Example insight: 'Lot A expected 87% full 08:00–10:00 tomorrow — consider overflow'

Target: MAPE target <12% · Weekly automated retraining

Component D — Dynamic Pricing Engine

Technology: Rule-based + ML-assisted pricing (Phase 3–4 optional feature)

Pipeline:

1. Base rate × demand multiplier (1.0× at <40% occupancy → 2.0× at >95% occupancy)
2. Multiplier updated every 15 minutes based on current occupancy + next-hour forecast
3. Price shown to driver before booking confirmation to encourage off-peak usage
4. Revenue uplift target: +20% in peak periods vs flat-rate pricing

Target: Revenue optimisation · Off-peak demand shifting

04 Database Schema Enhancements

4.1 New and Modified Models

Model	New Fields	Purpose
ParkingSpace	last_updated, sensor_id, camera_zone_id, confidence_score	Track which sensor/camera detected occupancy and with what confidence
ParkingTicket	booking_type (walk-in/reserved/ANPR), plate_confidence, gate_in_image, gate_out_image	Distinguish manual vs automated tickets; store entry/exit camera frames
ParkingLot	latitude, longitude, hourly_rate, currency, accepts_reservations, operating_hours	Enable map display, dynamic pricing, and time-based availability
Vehicle	last_seen_at, is_blacklisted, blacklist_reason	Operational tracking; security flag for vehicles of concern
Notification	user, message, type (push/email/sms), is_read, sent_at	In-app + push notification history per driver
MLPrediction	lot, prediction_date, hour, predicted_pct, actual_pct, model_version	Store forecasts and track model accuracy over time
AuditLog	user, action, target_model, target_id, ip_address, timestamp	Security and compliance audit trail for all admin actions
SensorReading	sensor_id, space (FK), is_occupied, raw_distance_cm, timestamp	Raw sensor telemetry for debugging and model training data

4.2 Migration Strategy (Zero Downtime)

Phase 1: Add nullable new fields to existing models — existing rows unaffected, no table lock

Phase 2: Backfill new fields from historical data where derivable

Phase 3: Create new tables: MLPrediction, SensorReading, AuditLog, Notification

Phase 4: Add DB indexes on high-query columns: entry_time, exit_time, is_occupied, parking_lot_id

Phase 5: Evaluate TimescaleDB extension for SensorReading time-series queries at scale

05 REST API Design

5.1 Public Endpoints — No Authentication Required

Method	Endpoint	Description
GET	`/api/lots/`	List all parking lots with real-time occupancy counts and coordinates
GET	`/api/lots/{id}/spaces/`	Available spaces in a specific lot with type and status

GET	/api/lots/{id}/predictions/	24-hour occupancy forecast for a lot (JSON array by hour)

5.2 Driver Endpoints — JWT Authentication Required

Method	Endpoint	Description
POST	/api/auth/login/	Returns JWT access + refresh tokens
POST	/api/auth/register/	Create driver account
POST	/api/bookings/	Reserve a parking space for a specific time window
GET	/api/bookings/active/	Current active booking with space details and QR code
POST	/api/bookings/{id}/cancel/	Cancel a reservation (refund logic TBD)
GET	/api/tickets/	Full ticket history with entry/exit times and fees
GET	/api/notifications/	In-app notification history (unread count in header)

5.3 Admin Endpoints — JWT + is_staff Required

Method	Endpoint	Description
GET	/api/admin/dashboard/	All dashboard statistics in one call
POST	/api/admin/anpr/process/	Submit camera frame (multipart) → returns plate + confidence
POST	/api/admin/spaces/{id}/occupancy/	Manual occupancy override with reason
GET	/api/admin/reports/revenue/	Revenue analytics with start/end date filters
GET	/api/admin/reports/predictions/	ML forecast accuracy history by lot
POST	/api/admin/vehicles/{id}/blacklist/	Flag vehicle with reason; creates AuditLog entry

5.4 WebSocket Events (Django Channels)

Real-time occupancy updates are pushed to all connected clients over WebSocket. No polling required — the browser updates the map and space grid instantly.

Channel URL	Event Payload
ws://parms.app/ws/lots/{lot_id}/occupancy/	{"space_id": 42, "is_occupied": true, "updated_at": "2025-05-04T08:15:00Z", "source": "camera", "confidence": 0.96}

06 Technology Recommendations

6.1 ML & Computer Vision

Technology	Purpose	Licence	Notes
YOLOv8 (Ultralytics)	Vehicle & plate detection	AGPL-3.0	State-of-the-art accuracy; runs on Jetson Nano; ONNX export for fast inference
EasyOCR	Plate text recognition	Apache 2.0	Supports 80+ languages including Kinyarwanda; easy Python integration
OpenCV	Image preprocessing	Apache 2.0	Industry standard; frame capture, cropping, brightness normalisation
Facebook Prophet	Occupancy forecasting	MIT	Robust to missing data and seasonality; weekly/daily trend decomposition
scikit-learn	Feature engineering, eval	BSD	Well-documented; used for MAPE calculation and model evaluation
ONNX Runtime	Optimised model inference	MIT	Deploy YOLOv8 as ONNX for 3-5× faster inference on CPU
Plate Recognizer API	Hosted ANPR fallback	Commercial	2,500 free lookups/month; >98% accuracy; used when local confidence < 0.70

6.2 Backend & Infrastructure

Technology	Purpose	Licence	Notes
Django REST Framework	REST API layer	BSD	Integrates cleanly with existing Django codebase; serializers + ViewSets
Django Channels + Daphne	WebSockets / ASGI	BSD	Real-time occupancy broadcast; drop-in ASGI server replacing Gunicorn
Celery + Redis	Async task queue	BSD / MIT	ML inference jobs, scheduled retraining, email/SMS dispatch
FastAPI	ML microservice	MIT	High-throughput async inference endpoint; separate from Django monolith
PostgreSQL 16	Primary database	PostgreSQL	Existing choice; add TimescaleDB extension for time-series sensor data
Redis 7	Cache + Celery broker	BSD	Real-time occupancy state, session cache, WebSocket pub/sub channel layer
Mosquitto (MQTT)	IoT sensor broker	EPL-2.0	Lightweight; widely supported in parking IoT hardware ecosystems

Docker + Compose	Containerisation	Apache 2.0	Consistent dev/prod environments; single docker-compose up to start stack
Nginx	Reverse proxy	BSD	Serves static files, SSL termination, load balancing, WebSocket proxying

6.3 Frontend & Mobile

Technology	Purpose	Licence	Notes
React.js	Driver web app	MIT	Component-based; rich ecosystem; reuse components in React Native
Leaflet.js + OpenStreetMap	Parking map	BSD / ODbL	Free; no API key required; works offline with cached tiles
Chart.js	Admin analytics charts	MIT	Lightweight; easy Django template integration; real-time data support
React Native	Mobile app (iOS + Android)	MIT	Single codebase; shares API client code with React web app
Firebase FCM	Push notifications	Commercial	Free tier: unlimited notifications; iOS + Android + web support

6.4 Cloud & Deployment

Technology	Purpose	Licence	Notes
Render.com	Web service + PostgreSQL	Commercial	Already in use; sufficient through Phase 2; ~$45/month combined
DigitalOcean Droplet	ML microservice hosting	Commercial	$24/month for 4GB RAM; GPU upgrade optional for heavy inference
Cloudflare R2 / AWS S3	Camera frame storage	Commercial	~$0.015/GB/month; gate images attached to tickets and deleted after 30d
Cloudflare	CDN + DDoS protection	Commercial	Free tier sufficient for PARMS scale; automatic HTTPS
GitHub Actions	CI/CD pipeline	Free	Auto-run tests and deploy to Render on push to main branch

07 Security & Compliance

7.1 Authentication & Authorisation

- Replace session auth with JWT (djangorestframework-simplejwt) for all API endpoints; keep session auth for the web dashboard
- Implement OAuth2 social login (Google) for driver-facing app to reduce sign-up friction
- Role hierarchy: Super Admin > Admin > Attendant > Driver — each with distinct permission set
- Attendant role: can view and manage tickets and spaces but cannot create/delete lots or manage users

7.2 Data Protection

- Encrypt sensitive fields at rest: vehicle plate, owner name, contact info (django-encrypted-fields)
- HTTPS enforced everywhere — already provided by Render.com and Cloudflare
- Camera images stored in private S3/R2 bucket — not publicly accessible; presigned URLs for admin view
- GDPR compliance: driver can request full account deletion; ticket data retained for 2 years then purged
- Blacklisted plates: every blacklist action creates an AuditLog entry with user, reason, and timestamp

7.3 API Security

- Rate limiting: django-ratelimit — 100 req/min per IP on public endpoints; 500 req/min for authenticated
- CORS: whitelist only known domains via django-cors-headers; reject all others
- ANPR endpoint: validate file size (<2MB), MIME type (image/jpeg, image/png), and minimum resolution
- SQL injection: already protected by Django ORM — maintain policy of never using raw queries with user input
- JWT token expiry: access token 15 minutes, refresh token 7 days; refresh rotation enabled

7.4 Operational Security

- AuditLog model records every admin action: what changed, who changed it, from which IP, at what time
- Failed login attempts tracked; account locked for 15 minutes after 5 consecutive failures
- Attendant accounts scoped — cannot access billing data, ML controls, or user management
- Periodic security review: dependency audit with pip-audit monthly; OWASP Top 10 review annually

08 Implementation Roadmap — 5 Phases

The roadmap is structured to deliver value incrementally, with each phase building on the previous. Phases 1–2 stabilise and extend the existing system. Phase 3 introduces ML. Phases 4–5 complete the platform with mobile, smart pricing, and scale.

Phase 1 — Stabilisation & API Foundation — Months 1–2

Goal: Production-ready, secure, and API-capable

Tasks:

1. Add Django REST Framework; expose read-only public API for lots, spaces, and predictions
2. Add JWT authentication for all API endpoints; social login for drivers
3. Database indexes and N+1 query audit across all views
4. Rate limiting (django-ratelimit), CORS (django-cors-headers), audit logging
5. Integrate Chart.js in admin dashboard — revenue trend, occupancy bar chart, ticket volume
6. Real email notifications via SendGrid: booking confirmation, ticket expiry warning
7. Docker Compose local environment (Django + PostgreSQL + Redis + Celery)
8. GitHub Actions CI: run test suite on every pull request

Deliverables:

- Stable REST API with JWT auth
- Email notifications
- Charts in admin dashboard
- Dockerised dev environment
- Automated test pipeline

Phase 2 — Real-time & Maps — Months 3–4

Goal: Live occupancy visible to drivers and admins without page refresh

Tasks:

1. Add Django Channels (ASGI) + Daphne; replace Gunicorn with ASGI-compatible server
2. WebSocket consumer per lot broadcasts occupancy state changes in real time
3. MQTT subscriber Celery worker ingests sensor data and updates ParkingSpace.is_occupied
4. Driver-facing map page with Leaflet.js + OpenStreetMap showing real-time space availability
5. Reservation workflow: driver books a specific space for a future time window
6. Push notifications via Firebase FCM: space available, booking confirmed, session expiry
7. Celery beat scheduled tasks: expiry reminders 30 min before, daily revenue summary email

Deliverables:

- Live occupancy map for drivers
- Sensor data ingestion pipeline
- Reservations workflow
- Push notifications
- Scheduled operational tasks

Phase 3 — ML Integration — Months 5–7

Goal: Automated vehicle detection, plate recognition, and occupancy prediction

Tasks:

1. Set up FastAPI ML microservice with YOLOv8 + EasyOCR; expose /anpr and /occupancy endpoints
2. Integrate camera feed processing pipeline: capture frame → detect vehicle → extract plate
3. Connect ANPR output to ticket creation (entry) and closure (exit) workflows
4. Deploy occupancy detection: camera-based polygon method OR sensor-based depending on hardware
5. Train initial Prophet forecasting model on historical ParkingTicket data
6. Surface predictions in admin dashboard: '24h forecast' chart per lot
7. Build model monitoring: log predicted vs actual occupancy weekly; alert if MAPE > 15%
8. Add Plate Recognizer API as cloud fallback when local model confidence < 0.70

Deliverables:

- Working ANPR at entry/exit gates
- Automated ticket lifecycle
- 24-hour occupancy forecast in dashboard
- Model accuracy monitoring
- Cloud ANPR fallback

Phase 4 — Mobile App & Advanced Features — Months 8–10

Goal: Full driver mobile experience and revenue optimisation

Tasks:

1. Build React Native mobile app (iOS + Android) consuming the REST API with JWT auth
2. In-app turn-by-turn navigation to parking space using Mapbox Directions API
3. In-app payment: Stripe for cards; MTN MoMo API for mobile money (Rwanda/Africa markets)
4. Dynamic pricing engine: base rate × demand multiplier; updated every 15 minutes
5. Driver loyalty programme: points earned per parking session; redeemable for discounts
6. Admin mobile view for on-the-go lot management and ticket oversight
7. Export reports to PDF and Excel from admin dashboard
8. Load testing with Locust: validate 500 concurrent users; optimise slow queries

Deliverables:

- Published iOS + Android app
- Real payment gateway integration
- Dynamic pricing live
- Driver loyalty programme
- Load-tested for 500 concurrent users

Phase 5 — Scale & Smart City Integration — Months 11–12

Goal: Multi-site management and open smart city data sharing

Tasks:

1. Multi-tenancy: support multiple parking operators on one platform (schema-per-tenant)
2. Open Data API: publish anonymised hourly occupancy statistics for smart city dashboards
3. TimescaleDB migration for SensorReading table — efficient time-range queries at million-row scale
4. Horizontal scaling: Kubernetes (AWS EKS) with auto-scaling based on request load
5. SLA monitoring: uptime, API response time P95, ML inference latency in Grafana dashboard
6. Third-party integrations: Google Maps Places API (show availability badge), Waze, Apple Maps

Deliverables:

- Multi-operator platform
- Smart city open data API
- TimescaleDB for sensor data
- Kubernetes auto-scaling
- Google Maps / Waze integration

09 Cost Estimates

9.1 Monthly Infrastructure Cost (Production)

Service	Cost (USD/month)	Notes
Render.com Web Service (Standard)	$25	Django app + Gunicorn/Daphne
Render.com PostgreSQL (Standard)	$20	Managed PostgreSQL 16
Redis Cloud (free → 100MB paid tier)	$0–$10	Celery broker + real-time cache
DigitalOcean Droplet (ML microservice)	$24	4GB RAM, 2 vCPU — FastAPI + YOLOv8
Cloudflare R2 (camera frame storage)	$5–$15	~$0.015/GB; 30-day retention policy
SendGrid (up to 40,000 emails/month)	$0	Free tier sufficient for initial scale
Firebase FCM (push notifications)	$0	Free for unlimited notifications
Plate Recognizer API (2,500 req/month)	$0	Free tier fallback for ANPR
Domain name	$1/month	~$12/year
TOTAL (Phases 1–3)	~$75–$95/month	Before mobile payment gateway fees

9.2 One-Time Hardware Cost (Per Parking Lot, Basic Setup)

Item	Cost (USD)	Notes
Raspberry Pi 4 (8GB RAM) + case + power	$80	Edge compute node for sensor ingestion
NVIDIA Jetson Nano (heavier ML inference)	$150	Optional — use if camera ANPR is local
Raspberry Pi Camera Module v3 (wide-angle)	$25 each	Entry/exit gate cameras
IP Fisheye Camera (PoE, IR, 180°)	$60–$120	Overhead lot coverage for occupancy
Ultrasonic Sensors HC-SR04 × 10 pack	$15	Per-space occupancy detection
Ethernet switch (PoE), cables, mount	$50	Infrastructure cabling
TOTAL (basic single-lot setup)	~$280–$440	Scales linearly with lot size

9.3 Development Effort Estimate (Person-Months)

Phase	Backend (months)	ML (months)	Frontend/Mobile (months)	Total
Phase 1 — Stabilisation & API	1.5	0	0.5	2

Phase 2 — Real-time & Maps	1.5	0	1.0	2.5
Phase 3 — ML Integration	1.0	2.0	0.5	3.5
Phase 4 — Mobile & Smart Pricing	1.0	0.5	3.0	4.5
Phase 5 — Scale & Smart City	2.0	0.5	0.5	3
TOTAL	7.0	3.0	5.5	15.5

10 Success Metrics & KPIs

10.1 Operational Metrics

Metric	Target	Notes
Ticket creation time (ANPR → ticket issued)	< 3 seconds	End-to-end from vehicle detection
ANPR plate recognition accuracy	> 94%	Standard daylight conditions
Occupancy update latency	< 2 seconds	Sensor/camera detection to dashboard
System uptime (monthly)	> 99.5%	< 3.6 hours downtime per month
API response time P95	< 300 ms	All authenticated endpoints
ML inference time (ANPR)	< 800 ms	Local ONNX model; cloud fallback < 1.5s

10.2 Business Metrics

Metric	Target	Notes
Manual attendant workload reduction	Target: -60%	Vs baseline pre-ANPR
Parking space utilisation improvement	Target: +15%	Vs baseline before predictive pricing
Driver search-to-park time	< 5 minutes	From app open to space located
Revenue increase (dynamic pricing periods)	Target: +20%	During peak-hour demand windows
Driver app 30-day retention	> 40%	Standard mobile app benchmark
Contact form / support ticket volume	Target: -30%	Self-service via app reduces queries

10.3 ML Model Quality Metrics

Metric	Target	Notes
ANPR accuracy (daylight)	> 94%	Character-level plate accuracy
ANPR accuracy (low-light, IR cameras)	> 85%	Night-time with IR illumination

Occupancy detection accuracy	> 97%	Camera-based, standard conditions
Occupancy forecast MAPE	< 12%	Mean Absolute Percentage Error (24h)
False positive: blacklisted vehicle	< 0.1%	Minimise wrongful alerts
Model drift detection threshold	MAPE > 15%	Triggers automated retraining alert

11 Risks & Mitigations

Risk	Likelihood	Impact	Mitigation Strategy
ANPR fails in poor lighting / weather / dirty plates	High	Medium	IR cameras at entry/exit; Plate Recognizer API cloud fallback; manual attendant override always accessible in dashboard
Sensor hardware failure (ultrasonic / camera)	Medium	High	Redundant sensors per space; camera-based backup if sensor fails; manual override in admin dashboard
ML model accuracy degrades over time (model drift)	Medium	Medium	Weekly automated retraining on fresh data; MAPE monitoring; Slack/email alert when MAPE > 15%
Database performance degradation under high concurrent load	Medium	High	DB indexes on hot columns; query optimisation (N+1 audit); read replica in Phase 5; Celery for heavy queries
Privacy / GDPR violation from camera data	Low	High	Camera images auto-deleted after 30 days; no facial recognition; clear privacy policy; GDPR deletion endpoint
Low driver adoption of web / mobile app	Medium	Medium	Web-first approach (no install required); first-park-free promo; QR code stickers at lot entrances
IoT MQTT broker security compromise	Low	High	TLS on all MQTT connections; device certificates for edge nodes; network isolation (VLAN) for IoT devices
Payment gateway failure during peak hours	Low	High	Multiple payment methods (MTN MoMo + Stripe + cash fallback); retry logic with exponential backoff

12 Conclusion

PARMS has a well-structured Django foundation with solid CRUD operations, role-based access control, a clean single-page admin interface, and a driver-facing dashboard. The system is production-ready at its current scope but falls significantly short of what a modern Smart Parking Management System requires to compete in a world of real-time urban mobility platforms.

The proposed v2.0 architecture closes this gap through five pragmatic phases: beginning with immediate high-value wins (REST API, real-time charts, email notifications) and progressing to ML-powered automation (ANPR, occupancy detection, demand forecasting), and eventually a full mobile experience with dynamic pricing and smart city integration.

The two highest-impact ML additions are **Automatic Number Plate Recognition (ANPR)** and **real-time occupancy detection**. Together, they eliminate the two most labour-intensive manual tasks in daily parking operations — vehicle check-in and space monitoring — enabling authorities to manage significantly more spaces with fewer staff, while giving drivers accurate, live information to make parking decisions in under 5 minutes.

Key Outcome	Target
Investment to reach Phase 3 (full ML)	~$75–95/month infrastructure + ~6.5 person-months development
Projected staff workload reduction	60% fewer manual attendant tasks
Revenue uplift (dynamic pricing)	+20% during peak demand windows
Driver app adoption target (30-day)	>40% retention after first use
System uptime target	>99.5% monthly

PARMS v2.0 is not merely a software upgrade — it is a platform for intelligent urban mobility. Built on a proven Django foundation and extended with open-source ML tools, real-time communication, and a mobile-first driver experience, it is positioned to serve as the operational backbone of modern smart city parking infrastructure.

A Appendix A — Current File Structure

```
myproject/
■■■ myproject/
■   ■■■ settings.py          Django settings — DEBUG-aware static storage
■   ■■■ urls.py              Root URL conf (includes params_app.urls)
■   ■■■ wsgi.py / asgi.py    WSGI and ASGI entry points
■■■ params_app/
■   ■■■ models.py            ParkingLot, Vehicle, ParkingSpace, ParkingTicket,
■   ■                       Payment, ContactMessage (7 models)
■   ■■■ views.py             All views — _admin_required decorator applied
■   ■■■ urls.py              30 URL patterns (public + admin CRUD)
■   ■■■ forms.py             ContactMessageForm
■   ■■■ signals.py           Django signals
■   ■■■ templates/
■   ■   ■■■ index.html       Homepage — hero, features, testimonials, footer
■   ■   ■■■ dashboard.html   Admin SPA — 8 tabs, modals, WebSocket-ready
■   ■   ■■■ userin.html      Driver dashboard — parking cards, activities
■   ■   ■■■ login.html       Login — extends index.html
■   ■   ■■■ signup.html      Sign-up — password strength indicator
■   ■   ■■■ contact.html     Contact form — saves to ContactMessage
■   ■   ■■■ billings.html    Billing history — real DB data
■   ■   ■■■ parkings.html    Parking overview — real DB data
■   ■   ■■■ slots.html       Space selection + payment method picker
■   ■   ■■■ ticket.html      Active ticket + QR code + history
■   ■   ■■■ admin/           Legacy sub-templates (redirected to SPA)
■   ■■■ static/
■       ■■■ css/natural_styles.css   Global styles (1,200 lines)
■       ■■■ js/                      JavaScript utilities
■       ■■■ imgs/                    Logo, parking lot photos, team photos
■■■ static/                  Global static (Bootstrap overrides)
■■■ staticfiles/             collectstatic output (production serving)
■■■ .env                     Local development settings (SQLite, DEBUG=True)
■■■ requirements.txt         Python dependencies
■■■ render.yaml              Render.com deployment configuration
```

B Appendix B — Recommended Python Packages for v2.0

Package	Purpose
djangorestframework >= 3.15	REST API framework — serializers, ViewSets, authentication
djangorestframework-simplejwt >= 5.3	JWT authentication for API endpoints
django-channels >= 4.0	WebSockets and ASGI support
channels-redis >= 4.2	Redis channel layer for Django Channels
daphne >= 4.1	ASGI server replacing Gunicorn
celery >= 5.3	Distributed async task queue
redis >= 5.0	Python Redis client for Celery + cache
fastapi >= 0.110	ML microservice framework — async, high-throughput

uvicorn >= 0.29	ASGI server for FastAPI microservice
ultralytics >= 8.1	YOLOv8 — vehicle detection and plate localisation
easyocr >= 1.7	Plate text recognition — 80+ languages
opencv-python >= 4.9	Image preprocessing and camera frame capture
onnxruntime >= 1.17	Optimised ONNX model inference (3-5x faster than PyTorch CPU)
prophet >= 1.1	Facebook Prophet — occupancy time-series forecasting
scikit-learn >= 1.4	Feature engineering, MAPE evaluation, model utilities
paho-mqtt >= 2.0	MQTT client for IoT sensor data ingestion
django-ratelimit >= 4.1	API rate limiting by IP or user
django-cors-headers >= 4.3	CORS policy management for API endpoints
sendgrid >= 6.11	Transactional email (booking confirmation, alerts)
firebase-admin >= 6.4	Firebase FCM push notifications (iOS + Android + web)
django-encrypted-fields >= 0.1	Encrypt sensitive model fields at rest
stripe >= 8.0	Card payment processing
locust >= 2.24	Load testing — validate 500 concurrent users
pytest-django >= 4.8	Django test runner for CI pipeline